| | |
|---|---|
| **Challenge:** | The Buried Spell |
| **Category:** | Steganography + Reverse Engineering |
| **Difficulty:** | Medium |
| **Tools Used:** | Terminal (Linux / Git Bash), find, steghide, strings, Decompiler (Dogbolt / Ghidra / IDA), objdump, Python 3 |

## Summary

This challenge demonstrates how steganography can be combined with reverse engineering. By solving an acrostic cipher to deduce a passphrase, we extract a hidden executable embedded inside an image file. Reverse engineering the extracted binary reveals a hardcoded XOR decoding routine. By identifying the key and extracting the encoded data section from the binary, we can manually decrypt the payload to reveal the final flag.

## Problem Statement

No spell is forgotten - only buried, waiting for the right hand to unearth it .Evil grows where courage falters, where laughter is silenced by fear. Vanished things leave marks on the world - shadows where light once stood. In the darkest cupboards, monsters wear familiar faces. Laughter, they say, is the only magic that can unmake dread. Let the spell be spoken - and watch the dark thing crumble. Every hero is forged not in victory, but in the moment they choose to face what terrifies them most.
P.S. The first will show you the way.
FLAG FORMAT: CTF{}

## Steps to Solve:

1. Attempt to locate the challenge file in the default directory: **cd ~/Downloads**
2. The initial attempt fails. To locate the file, use the find command to search the entire home directory for the target image hpctf.jpg: **find ~ -name "hpctf.jpg"**
3. The search returns the file path: /path/to/your/file/. Navigate to this directory: **cd ~/path**
4. Analyze the image to check for embedded data using steghide: **steghide info hpctf.jpg**

```
ghxst0sint@Akshay:/mnt/c/Users/aksha/Downloads$ steghide info hpctf.jpg
steghide extract -sf hpctf.jpg
"hpctf.jpg":
  format: jpeg
  capacity: 93.6 KB
Try to get information about embedded data ? (y/n) y
Enter passphrase:
  embedded file "protected_spell.exe":
    size: 40.7 KB
    encrypted: rijndael-128, cbc
    compressed: yes
Enter passphrase:
wrote extracted data to "protected_spell.exe".
```

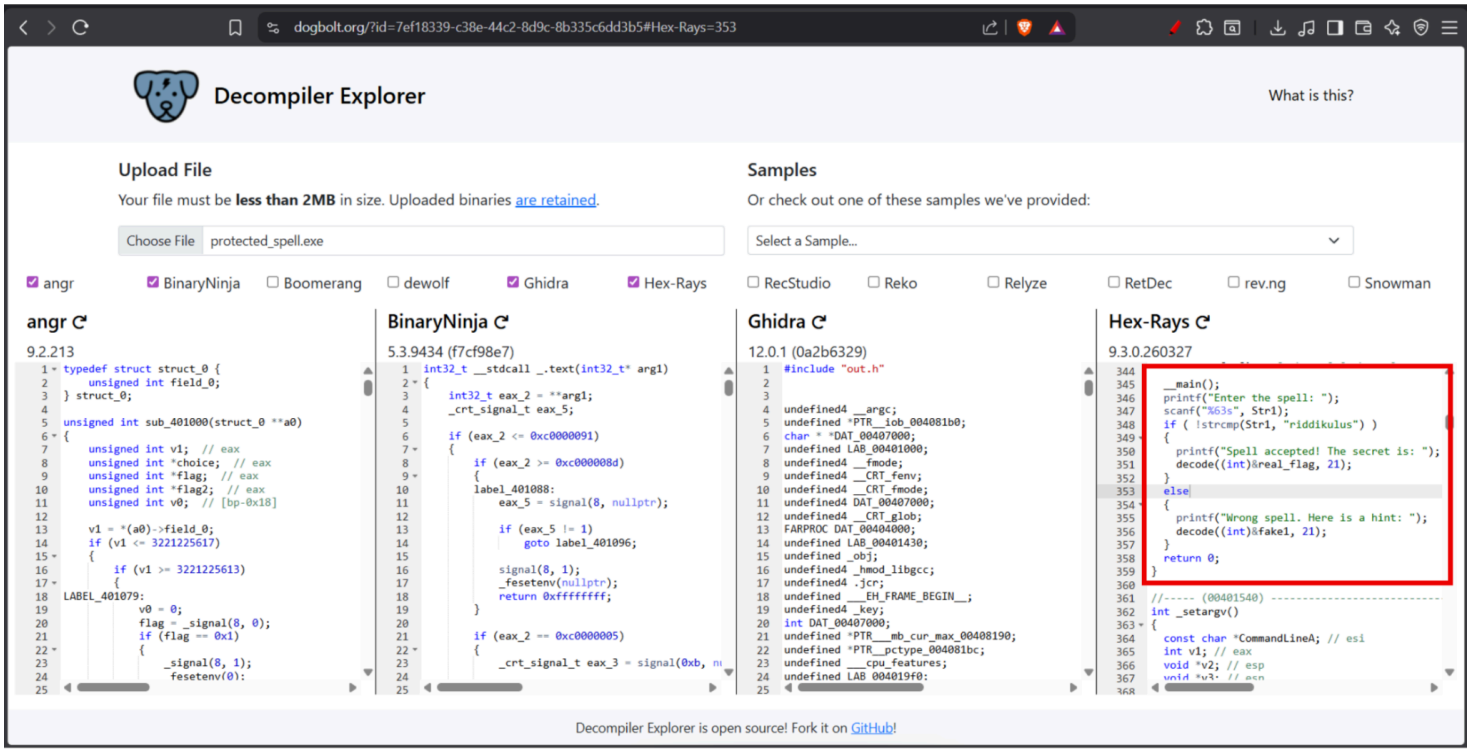
5. The output reveals that hidden embedded data exists within the image, but it requires a passphrase to extract.

6. To find the passphrase, refer to the challenge description: *"The first will show you the way"*. This suggests the use of an acrostic cipher.
7. Taking the first letter of each sentence from the broader challenge hints spells out the name: **NEVILLE**.
8. Using the thematic context of Harry Potter, Neville Longbottom fights a boggart using a specific charm. That spell is **riddikulus**, which acts as our passphrase.
9. Extract the hidden file using the discovered passphrase: **steghide extract -sf hpctf.jpg**
10. When prompted, enter the passphrase: riddikulus.
11. The extraction is successful, revealing a binary executable named protected_spell.exe.
12. Begin the initial binary analysis by searching for readable strings, specifically filtering for the word "flag":
    **strings protected_spell.exe | grep flag**

```
ghxst0sint@Akshay:/mnt/c/Users/aksha/Downloads$ strings protected_spell.exe | grep flag
_flag
__loader_flags__
_real_flag
```

13. The output returns _flag, __loader_flags__, and _real_flag. While these indicate we are on the right track, they are not the actual flag.
14. Decompile the binary (using a tool like Dogbolt) to reverse engineer its logic. Examining the main function reveals:

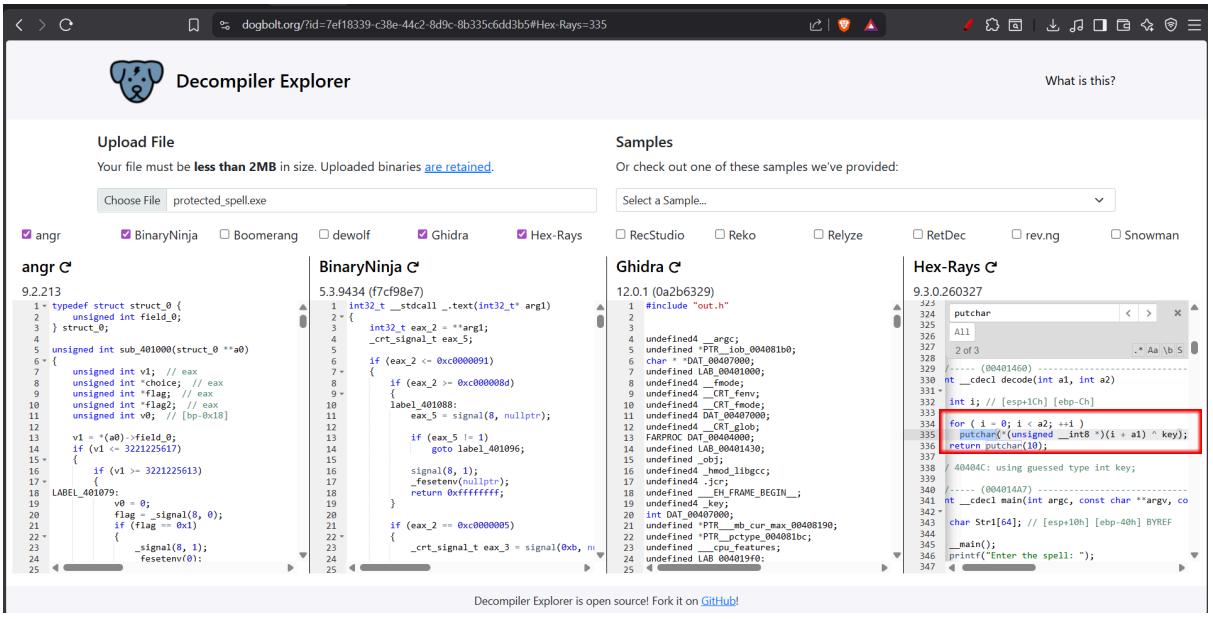
```
if (!strcmp(Str1, "riddikulus"))
{
decode((int)&real_flag, 21);
}
```




This confirms that providing the correct spell triggers the decoding of real_flag.

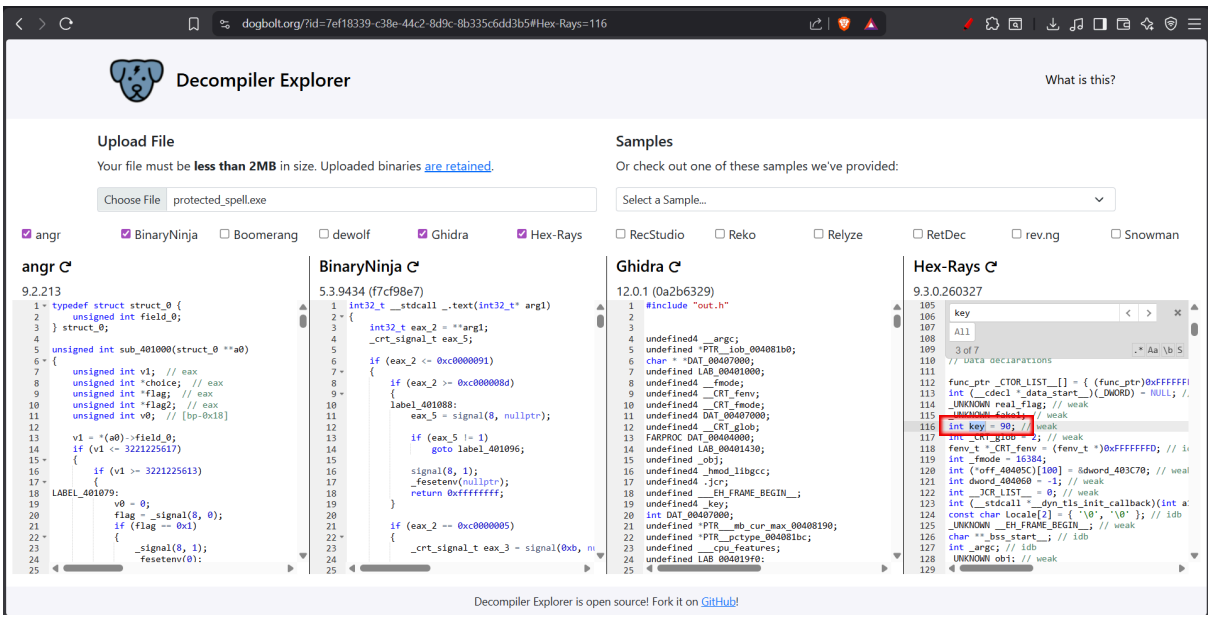
15. Analyze the decode function to understand the encryption method:

```
int __cdecl decode(int a1, int a2)
{
for (i = 0; i < a2; ++i)
putchar(*(unsigned __int8 *)(i + a1) ^ key);
}
```




This loop shows that the data is being obfuscated using an XOR cipher against a specific key.

16. Identify the key from the decompiled source code, which defines int key = 90;. Converting 90 to hexadecimal gives us 0x5A.



17. Next, extract the encoded data from the binary's .data section using objdump:
**objdump -s -j .data protected_spell.exe**

```
ghxst0sint@Akshay:/mnt/c/Users/aksha/Downloads$ objdump -s -j .data protected_spell.exe

protected_spell.exe:     file format pei-i386

Contents of section .data:
 404000 00000000 190e1c21 32352839 282f2205  .......!25(9(/".
 404010 2f34293f 3b363f3e 27000000 190e1c21  /4)?;6?>'......!
 404020 362f3735 2905373b 2233373b 056b6869  6/75).7;"37;.khi
 404030 27000000 190e1c21 3f222a3f 3636333b  '......!?"*?663;
 404040 28372f29 056e6f6c 27000000 5a000000  (7/).nol'...Z...
 404050 02000000 fdffffff 00400000 703c4000  .........@..p<@.
 404060 ffffffff 00000000                    ........
```

18. Locate the encoded flag bytes (e.g., around address 404004). The extracted hexadecimal data is: [0x19, 0x0e, 0x1c, 0x21, 0x32, 0x35, 0x28, 0x39, 0x28, 0x2f, 0x22, 0x05, 0x2f, 0x34, 0x29, 0x3f, 0x3b, 0x36, 0x3f, 0x3e, 0x27]
19. Finally, decode the extracted data. A Python 3 one-liner can be used to iterate through the array, XOR each byte with the key (0x5A), and print the resulting characters:
python3 -c "data=[0x19,0x0e,0x1c,0x21,0x32,0x35,0x28,0x39,0x28,0x2f,0x22,0x05,0x2f,0x34,0x29,0x3f,0x3b,0x36,0x3f,0x3e,0x27]; key=0x5A; print(''.join(chr(x ^ key) for x in data))"

```
ghxst0sint@Akshay:/mnt/c/Users/aksha/Downloads$ python3 -c "data=[0x19,0x0e,0x1c,0x21,0x32
,0x35,0x28,0x39,0x28,0x2f,0x22,0x05,0x2f,0x34,0x29,0x3f,0x3b,0x36,0x3f,0x3e,0x27]; key=0x5
A; print(''.join(chr(x ^ key) for x in data))"
CTF{horcrux_unsealed}
```

20. The script successfully reverses the XOR operation, outputting the final readable flag.
**Final FLAG:** CTF{horcrux_unsealed}